

Arhitectura calculatoarelor

Curs 11

11. Memorii

Cristian Vancea

<https://users.utcluj.ro/~vcristian/AC.html>

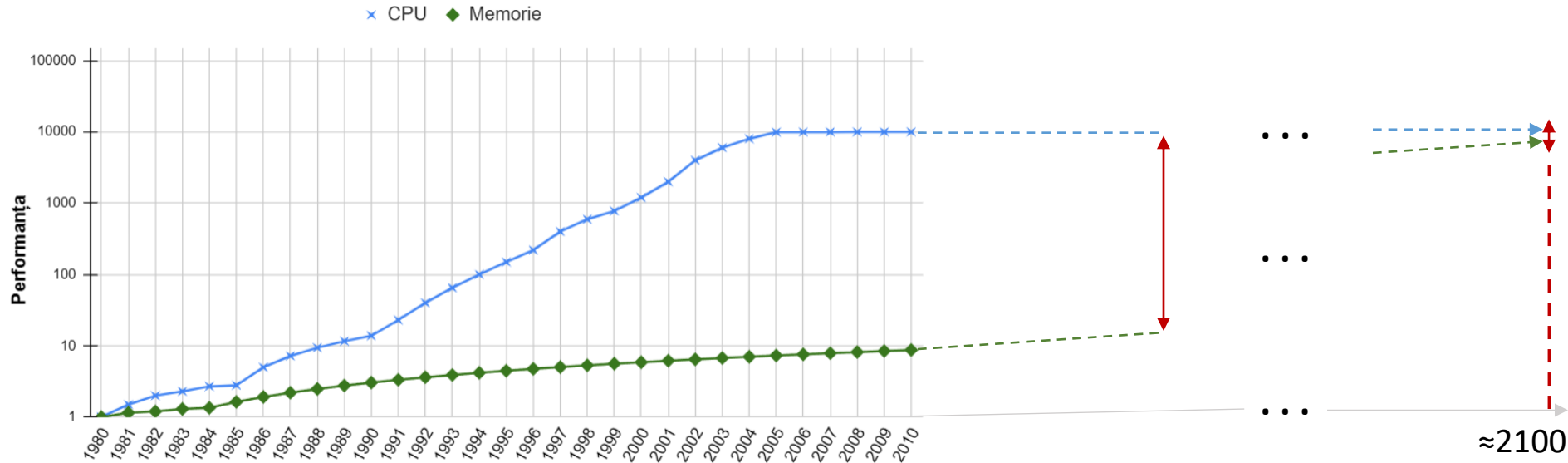
Objective

1. Organizarea ierarhică a memoriei
2. Memoria cache
 - 2.1 Plasarea blocului
 - 2.2 Identificarea blocului
 - 2.3 Înlocuirea blocului
 - 2.4 Modalități de scriere
 - 2.5 Performanța
 - 2.6 Conexiunea cu memoria
3. Memoria virtuală

11.1 Organizarea ierarhică a memoriei

11.1.1 Motivație

- Există o discrepanță între capacitatea procesorului de a accesa memoria și viteza de lucru a memoriei.



Performanța procesorului (1 nucleu) vs. memoria DRAM

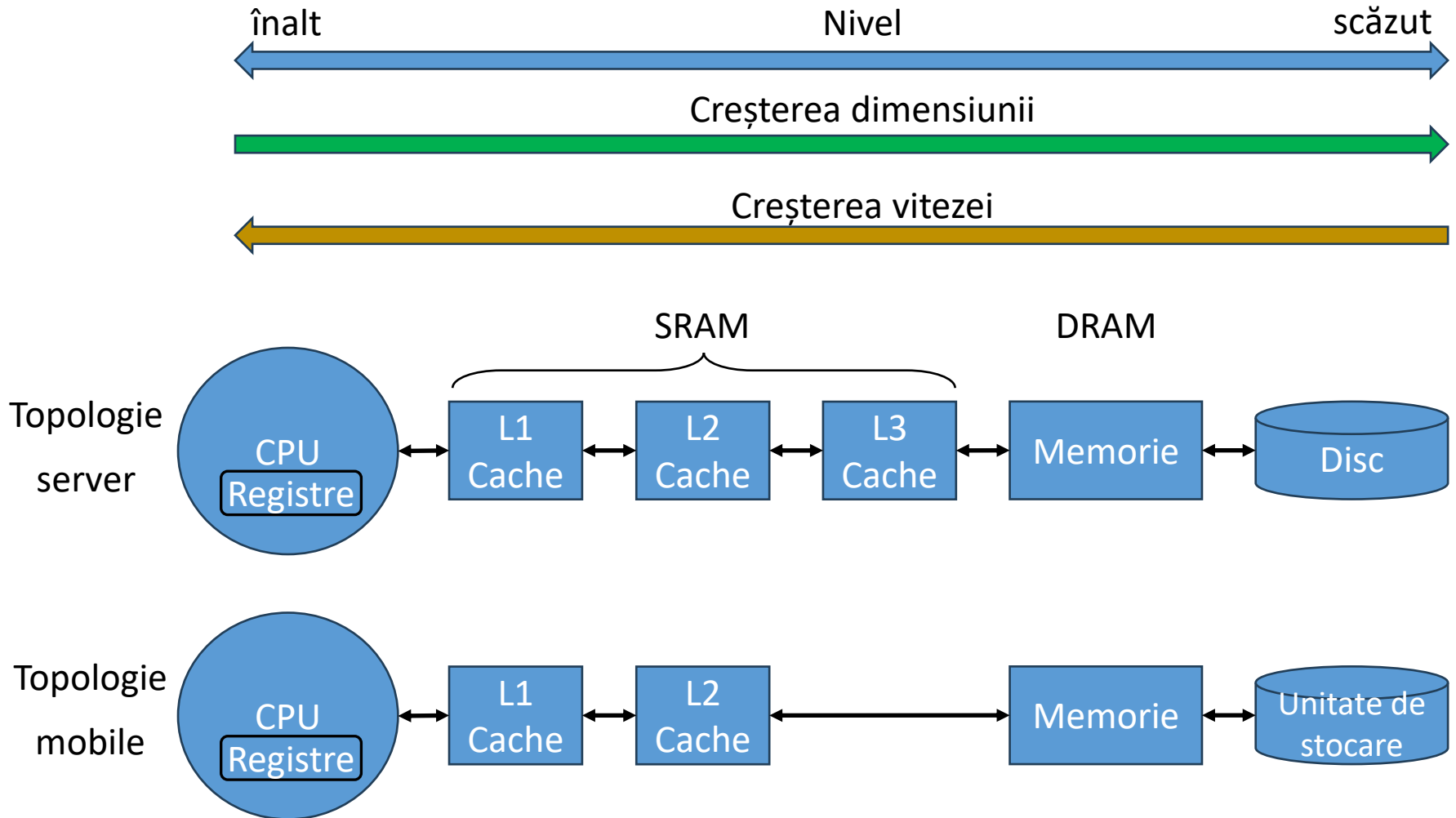
Performanța ridicată implică și costuri mari.



Tehnologie	Timp acces	Cost / GB [în 2012]
SRAM	0.5 – 2.5 ns	\$500 – \$1000
DRAM	50 – 70 ns	\$10 – \$20
Memorie Flash	5,000 – 50,000 ns	\$0.75 – \$1.00
Disc magnetic	5,000,000 – 20,000,000 ns	\$0.05 – \$0.10 ³

11.1 Organizarea ierarhică a memoriei

11.1.2 Soluție cu memorii cache și niveluri ierarhice



11.1 Organizarea ierarhică a memoriei

11.1.2 Soluție cu memorii cache și niveluri ierarhice – principii de funcționare

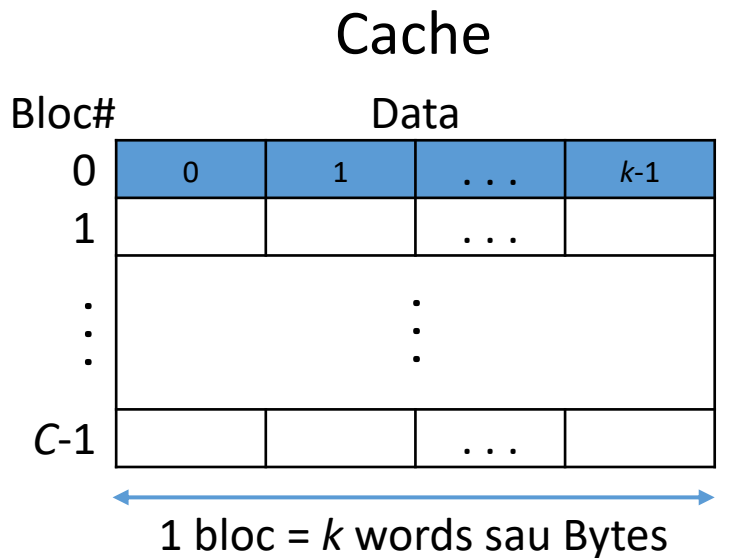
Ex: for (i=0; i<100; i++) x[i]+=s;

- Are la bază principiul localității:
 - **Localitate temporală** = o locație accesată recent este probabil să fie reacesată curând (ex: variabila s).
 - **Tehnica:** datele accesate recent sunt ținute la nivel înalt (aproape de procesor).
 - **Localitate spațială** = accesul la o adresă este probabil să fie urmată de accesuri la adrese vecine (ex: elementele x[i] și variabila s).
 - **Tehnica:** între memorii de niveluri adiacente se transferă **blocuri** de date care se întind pe mai multe adrese succesive.
- **Rezultat:** Se creează **iluzia** unei memorii la fel de rapidă ca și cea de nivel înalt și la fel de mare ca și cea de nivel scăzut, cu un cost financiar redus.

11.2 Memoria cache

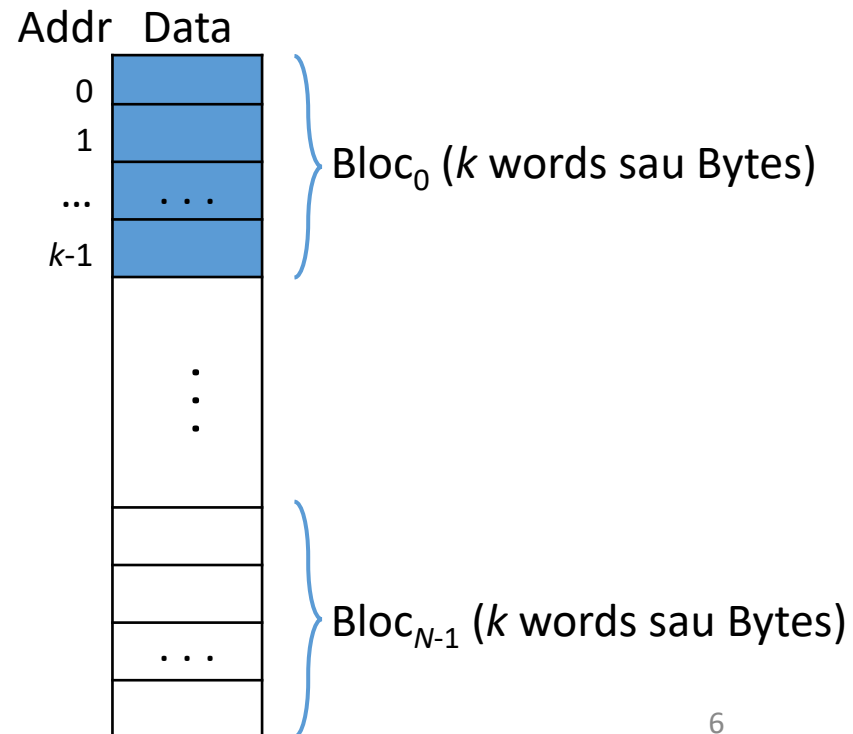
Termeni specifici utilizării memoriilor cache

- **Bloc** (block) sau **linie** (line) – unitatea minimă de informație luată în considerare la accesarea datelor dintr-un cache.
 - **Notă:** Chiar dacă în cache se poate accesa până la nivel de octet, transferul între cache și memoria de nivel inferior se realizează la nivel de blocuri, deoarece **memoriile sunt optimizate pentru transferuri în bloc**.



Cache size < Main Memory size
($C < N$)

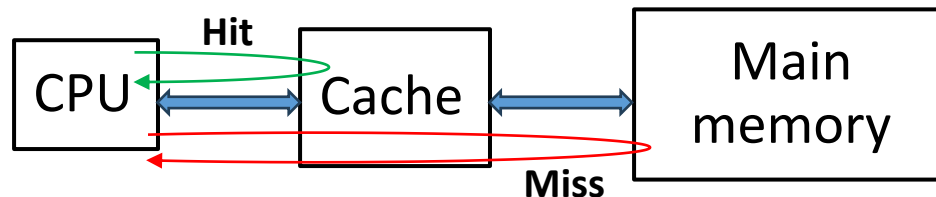
Main Memory



11.2 Memoria cache

Termeni specifici utilizării memoriilor cache (cont)

- **Hit** – data accesată se află într-un bloc din cache.
- **Miss** – data accesată nu se află într-un bloc din cache.
- **Hit rate** – rata de *hit* la memorie.
- **Miss rate** – rata de *miss* la memorie => $Miss\ rate = 1 - Hit\ rate$.
- **Hit time** – timpul necesar pentru a determina dacă accesul este *hit/miss* + timpul de returnare a datelor de la cache, odată ce sunt disponibile.
- **Miss penalty** – timpul adițional necesar pentru aducerea unui bloc de la memorie de nivel inferior la nivelul curent, în caz de *Miss*. $Miss\ penalty = \text{timp acces} + \text{timp înlocuire bloc} + \text{timp livrare la CPU}$.



11.2.1 Plasarea blocului

Modalități de organizare

- Mapată direct
- Direct Mapped**: fiecare bloc din memorie are o singură locație corespunzătoare în cache.
Calcul index bloc:

$$\text{Block_Address} \% \text{Cache_Blocks_No}$$

- Total asociativă
- Fully Associative**: un bloc din memorie poate fi plasat oriunde în cache.

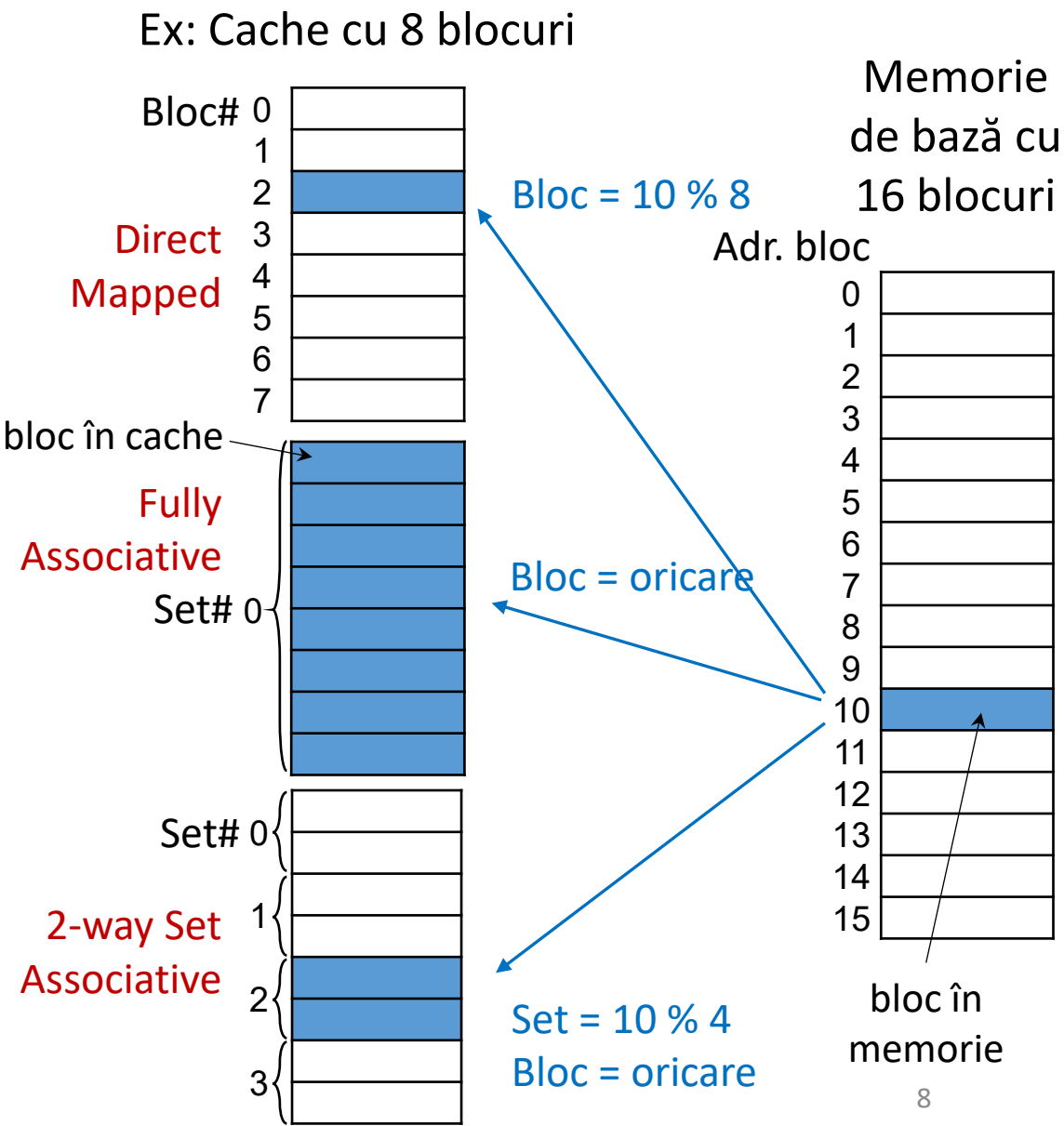
- Asociativă pe set
- Set Associative**: un bloc din memorie poate fi plasat la orice locație din cele n ale unui set (n -way Set Associative).

Calcul index set:

$$\text{Block_Address} \% \text{Cache_Sets_No}$$

Obs:

- Direct mapped $\Leftrightarrow$ 1-way Set Associative
- Fully Associative $\Leftrightarrow$ Set Associative cu 1 Set



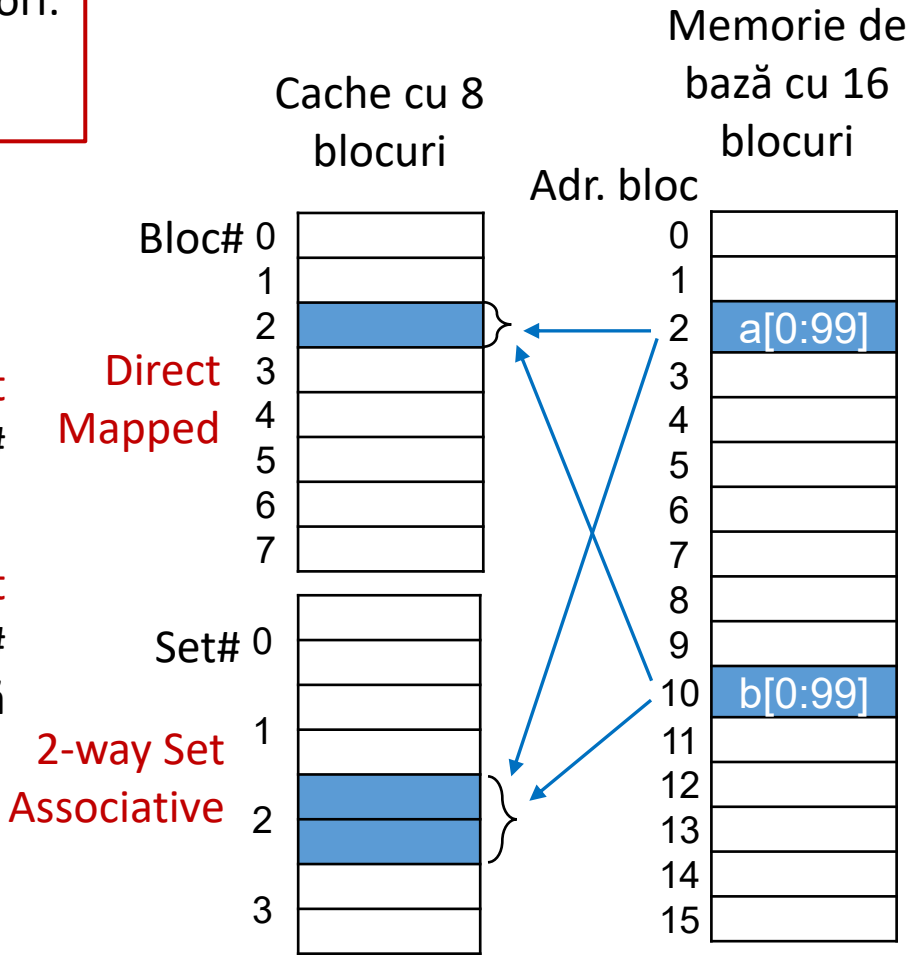
11.2.1 Plasarea blocului (cont)

Motivație Set Associative vs. Direct Mapped

```
Ex: Calculul produsului scalar dotp între vectori:  
for (dotp=0, i=0; i<100; i++)  
    dotp += a[i]*b[i];
```

Dacă în Memoria de bază $\text{Adr_bloc}(a[0:99])=2$
și $\text{Adr_bloc}(b[0:99])=10 \Rightarrow$

- Miss la fiecare iterație de buclă cu **Direct Mapped Cache**, datorat conflictului pe Bloc# 2 din cache.
- Hit la fiecare iterație de buclă cu **2-Way Set Associative Cache**, datorită faptului că Set# 2 din cache poate stoca 2 blocuri fără conflict.



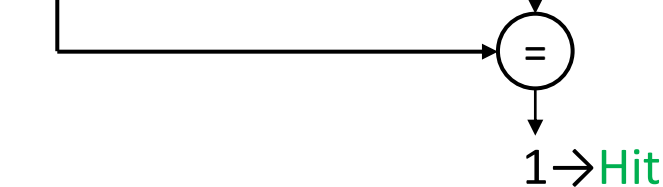
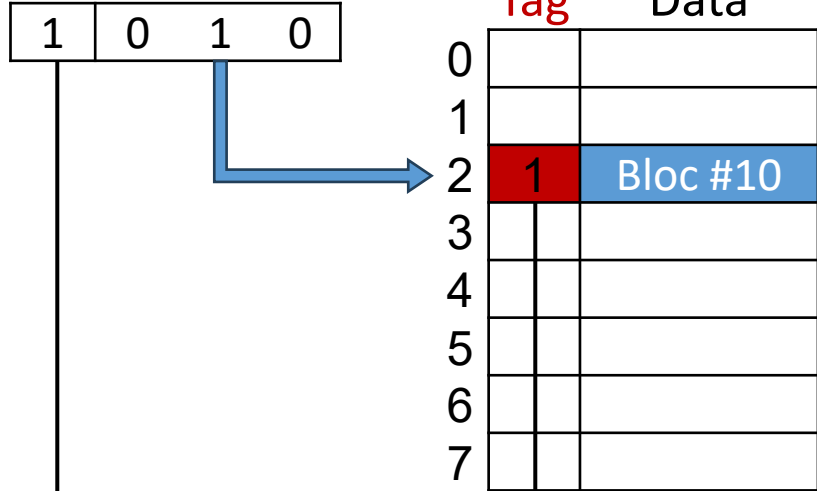
11.2.1 Plasarea blocului (cont)

Căutarea unui bloc în cache

- Întotdeauna numărul de blocuri/seturi în cache este putere a lui 2 $\Leftrightarrow 2^k$. Indexul blocului este dat de k biți cei mai puțin semnificativi din Adresa blocului.
- **Soluție:** Restul biților mai semnificativi $[msb:k+1]$ se salvează în cache (câmpul **Tag**) alături de conținutul blocului, pentru identificare unică.

Ex: Cache cu 8 blocuri

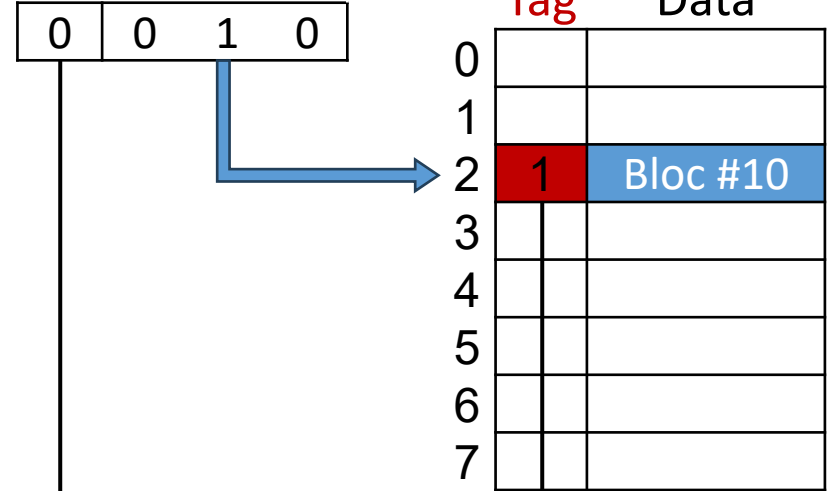
Adresă bloc = 10₁₀



tag index

10₁₀ = 1 0 1 0₂

Adresă bloc = 2₁₀

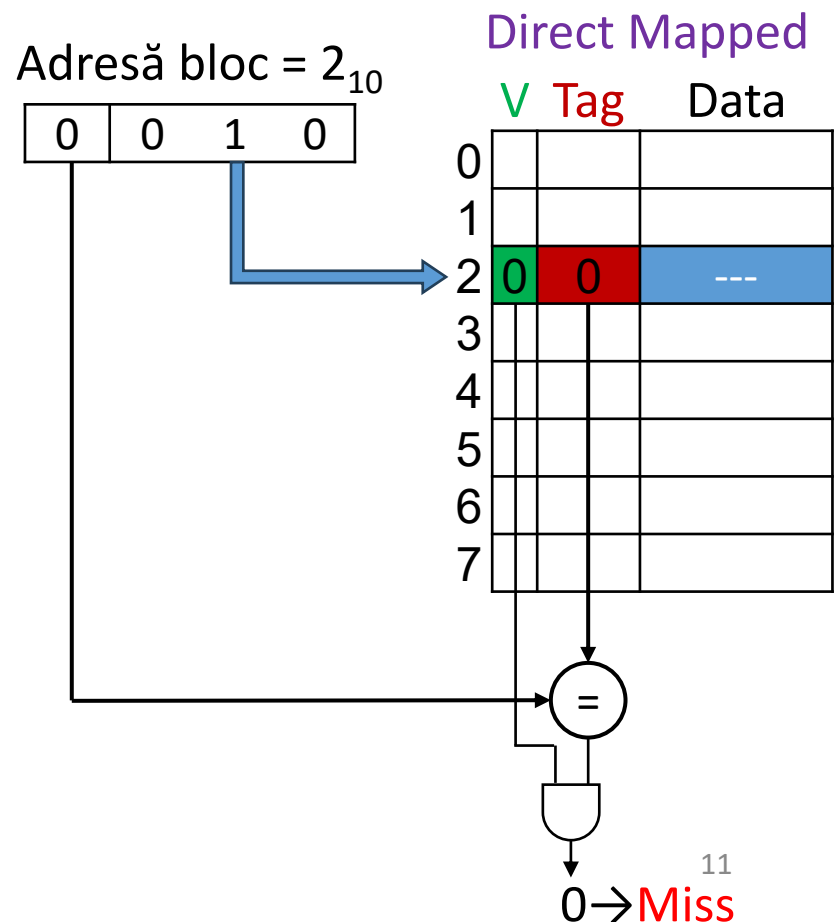
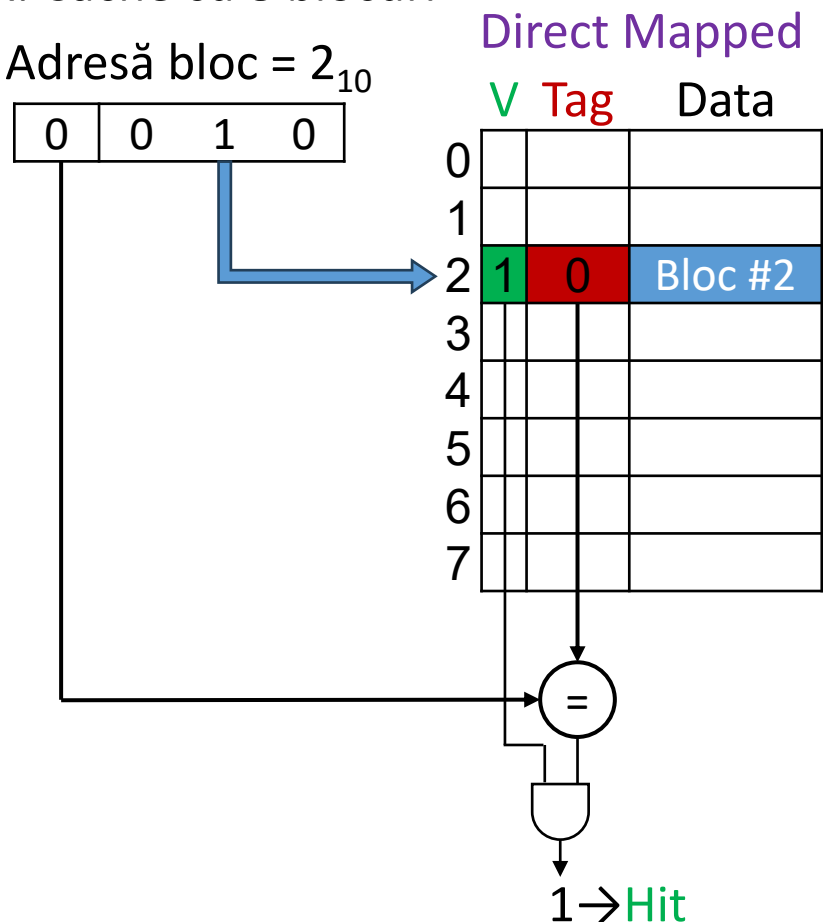


11.2.1 Plasarea blocului (cont)

Validitatea unui bloc în cache

- La inițializare, toate blocurile din cache conțin date invalide => se adaugă 1 bit adițional (câmpul **V** – valid) asociat blocului, care indică validitatea. Rezultatul final poate fi **Hit** sau **Miss**.

Ex: Cache cu 8 blocuri



11.2.1 Plasarea blocului (cont)

Exemple de configurații cache cu 8 blocuri

Bloc	V	Tag	Data
0			
1			
2			
3			
4			
5			
6			
7			

Way: 1
Direct Mapped
(1-way Set Associative)

Set	V	Tag	Data	V	Tag	Data
0						
1						
2						
3						

Way: 1 2
2-way Set Associative

Set	V	Tag	Data	V	Tag	Data	V	Tag	Data	V	Tag	Data
0												
1												

Way: 1 2 3 4
4-way Set Associative

V	Tag	Data	V	Tag	Data	V	Tag	Data	V	Tag	Data	V	Tag	Data	V	Tag	Data	V	Tag	Data

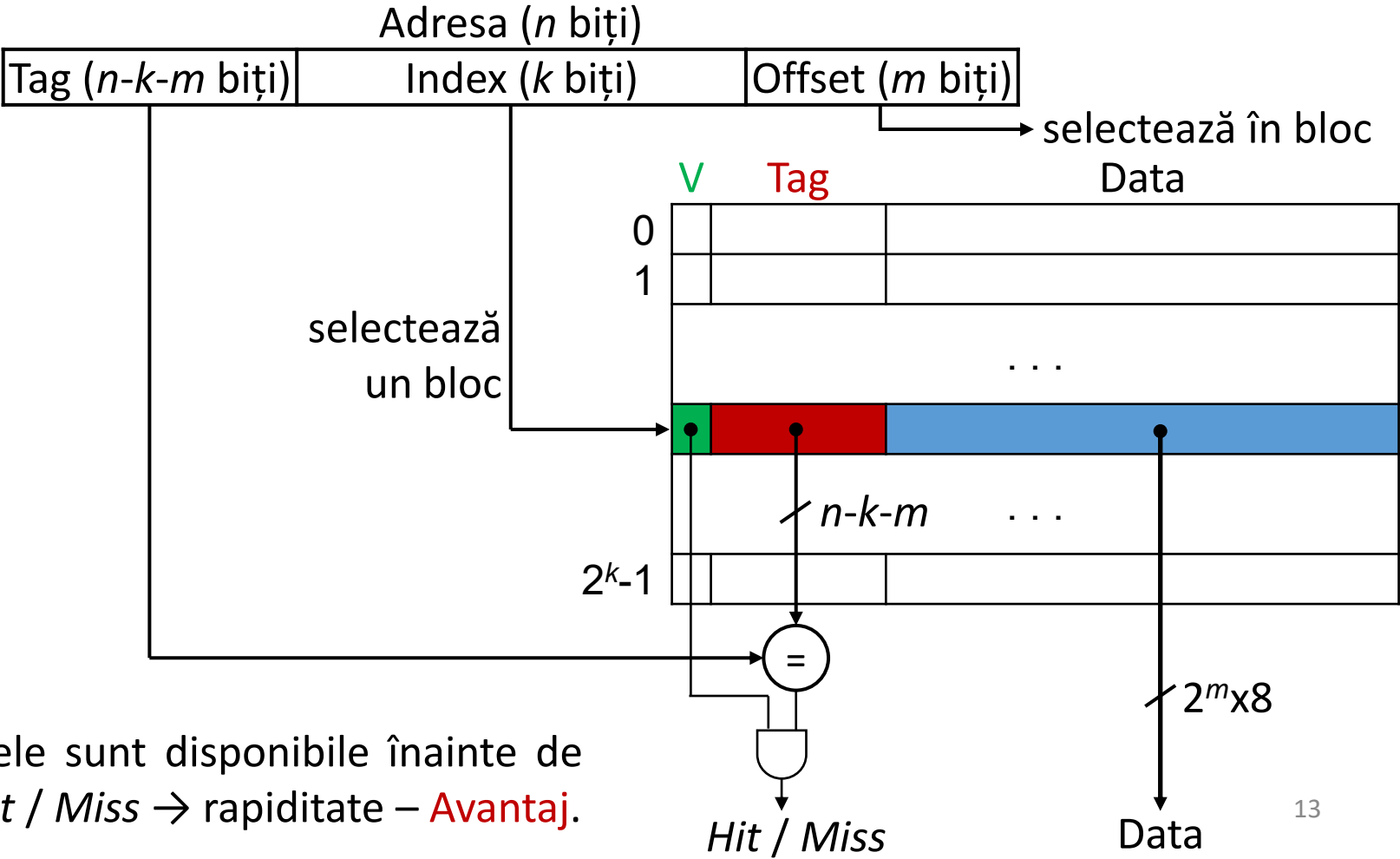
Way: 1 2 3 4 5 6 7 8
Fully Associative (8-way Set Associative)

11.2.2 Identificarea blocului

- Un cache interpretează adresa de memorie folosind câmpuri semnificative:



11.2.2.1 Identificarea blocului la Direct Mapped Cache

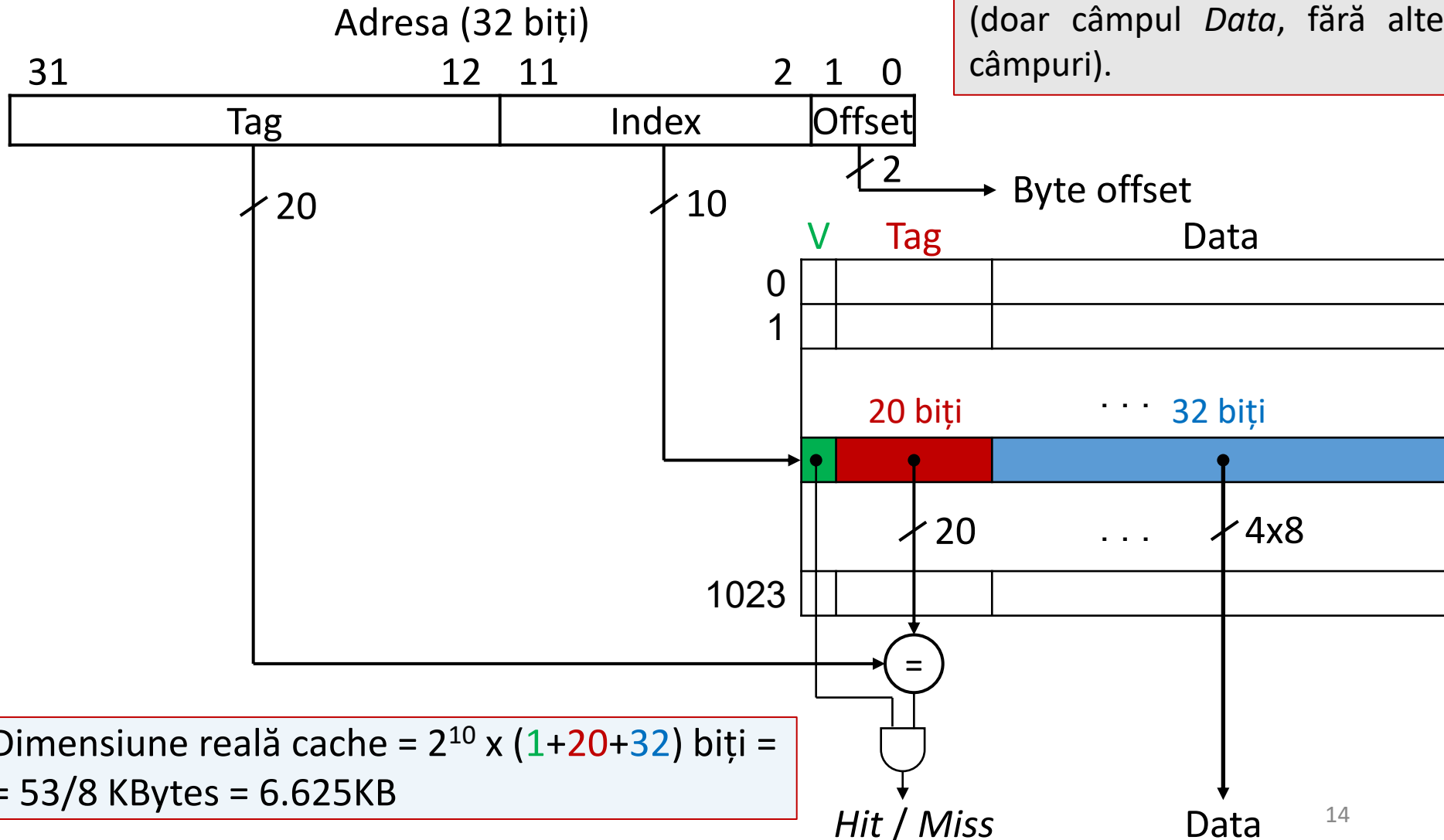


Obs: Datele sunt disponibile înainte de decizia *Hit / Miss* → rapiditate – **Avantaj**.

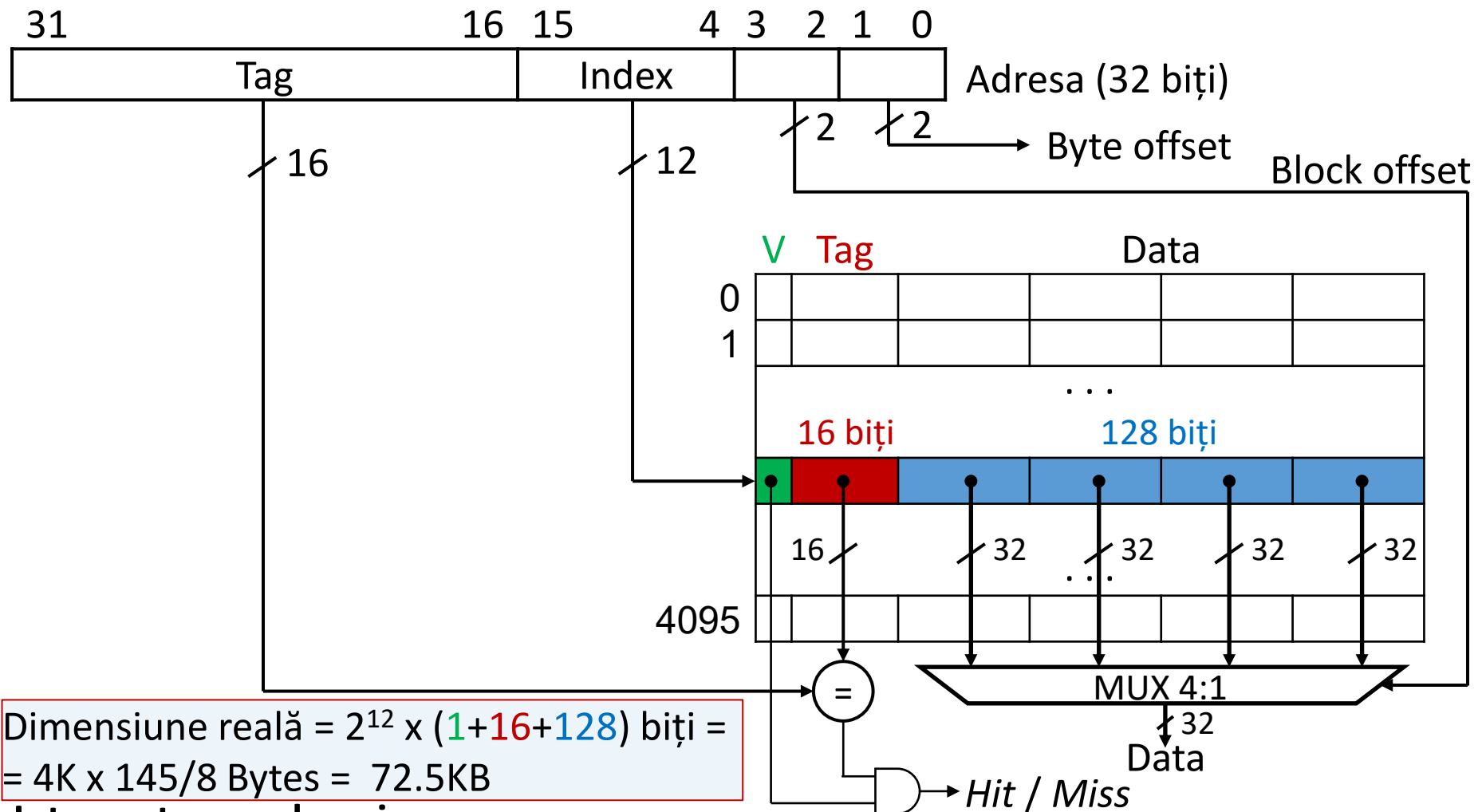
Ex₁: Direct Mapped Cache cu 1K (2^{10}) blocuri
 1 bloc = 1 cuvânt = 4 (2^2) Bytes } = 4KB Cache



Notă: Capacitatea unui cache se exprimă, de obicei, doar prin cantitatea de date memorată (doar câmpul *Data*, fără alte câmpuri).



Ex₂: Direct Mapped Cache de 64KB cu 4K (2^{12}) blocuri x 4 cuvinte/bloc, 1 cuvânt = 4 Bytes



Dimensiune reală = $2^{12} \times (1 + 16 + 128)$ biți =
 = 4K x 145/8 Bytes = 72.5KB

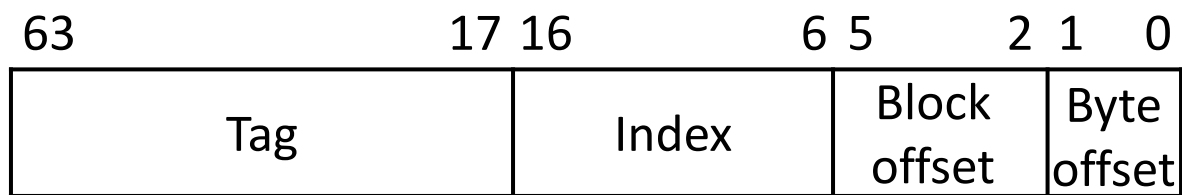
Interpretarea adresei:

- Byte offset (2 biți) – este ignorat de cache => returnează cuvinte de 4 Bytes.
- Block offset (2 biți) – adresează cuvântul din bloc.
- Index (12 biți) – adresează blocul în cache (sunt 2^{12} blocuri).
- Tag (16 biți) – restul de biți completează identificarea unică a blocului.

Problema₁: Care este dimensiunea câmpurilor dintr-o adresă de 64 biți dacă se folosește Direct Mapped Cache de 128KB cu 16 cuvinte / bloc?

Rezolvare:

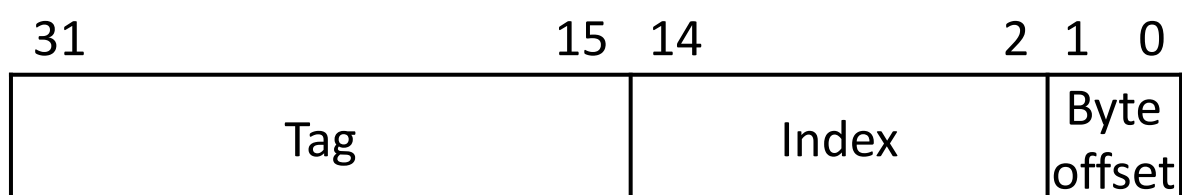
Dim. bloc = 16 x 4 Bytes = 2⁶ Bytes => Dim. Offset = 6 biți din care 2 sunt Byte offset.
Nr. blocuri = Dim. cache / Dim. bloc = 128K / 2⁶ = 2¹⁷ / 2⁶ = 2¹¹ => Dim. Index = 11 biți
Dim. Tag = Dim. adresă – Dim. Index – Dim. Offset = 64 – 11 – 6 = 47 biți



Problema₂: Care este dimensiunea câmpurilor dintr-o adresă de 32 biți dacă se folosește Direct Mapped Cache de 32KB cu 1 cuvânt / bloc?

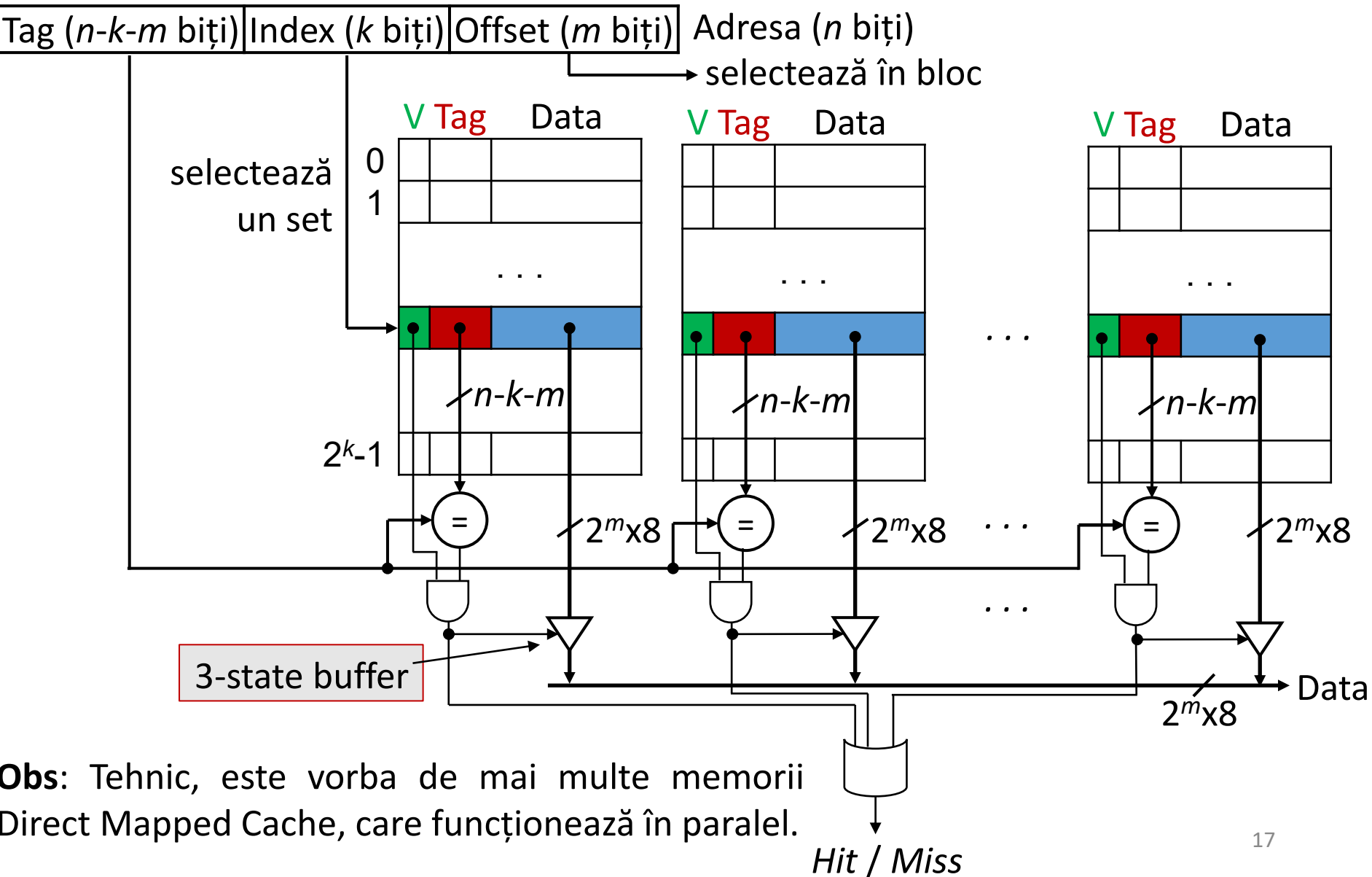
Rezolvare:

Dim. bloc = 4 Bytes = 2² Bytes => Dim. Offset = 2 biți de Byte offset.
Nr. blocuri = Dim. cache / Dim. bloc = 32K / 2² = 2¹⁵ / 2² = 2¹³ => Dim. Index = 13 biți
Dim. Tag = Dim. adresă – Dim. Index – Dim. Offset = 32 – 13 – 2 = 17 biți



11.2.2 Identificarea blocului (cont)

11.2.2.2 Identificarea blocului la Set Associative Cache



Obs: Tehnic, este vorba de mai multe memorii Direct Mapped Cache, care funcționează în paralel.

11.2.2 Identificarea blocului (cont)

11.2.2.3 Caracteristici pentru Set Associative Cache vs. Direct Mapped Cache

- **Avantaj:** Crește *Hit rate* pe măsură ce crește asociativitatea, dacă se păstrează dimensiunea cache => scade numărul de *Conflict Misses* (conflicte datorate aceluiași *Index* în adresă).
- **Dezavantaje:**
 1. Crește *Miss penalty* datorită strategiei mai complexe de înlocuire a blocului în caz de *Miss* (descrisă într-un slide următor);
 2. Număr ridicat de comparatoare;
 3. Necesită buffere 3-state suplimentare, care sunt mai lente;
 4. Mai lent – datele nu sunt disponibile decât după selecția setului, deci după decizia *Hit / Miss* pe căi (*ways*).

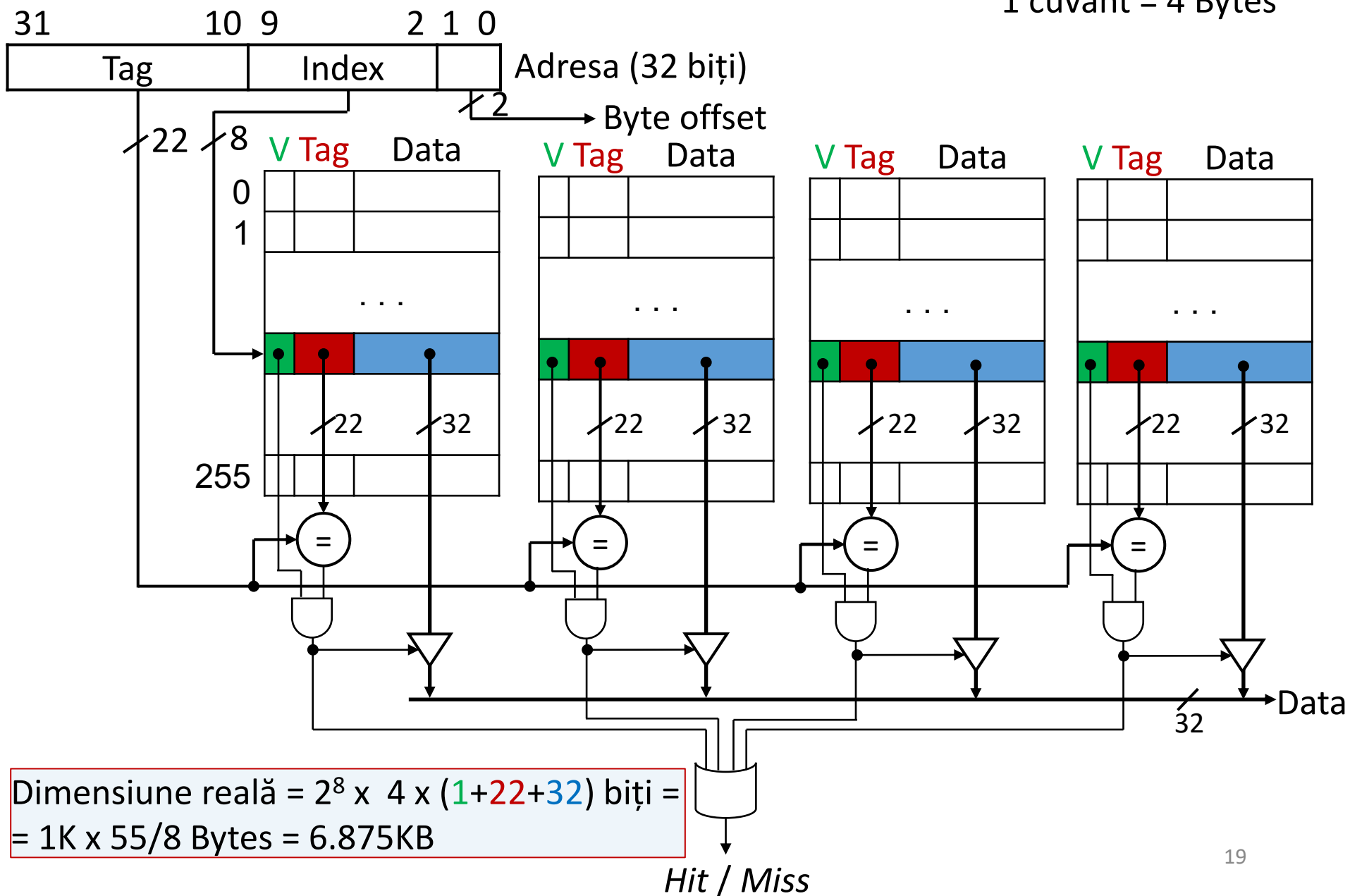
Problemă: Care este dimensiunea câmpurilor dintr-o adresă de 64 biți dacă se folosește 2-way Set Associative Cache de 1MB cu 4 cuvinte / bloc?

Rezolvare:

Dim. bloc = 4 x 4 Bytes = 2⁴ Bytes => Dim. Offset = 4 biți din care 2 sunt Byte offset.
Nr. seturi = Dim. cache / (Dim. bloc x Nr. Căi) = 2²⁰ / (2⁴ x 2) = 2¹⁵ => Dim. Index = 15 biți
Dim. Tag = Dim. adresă – Dim. Index – Dim. Offset = 64 – 15 – 4 = 45 biți

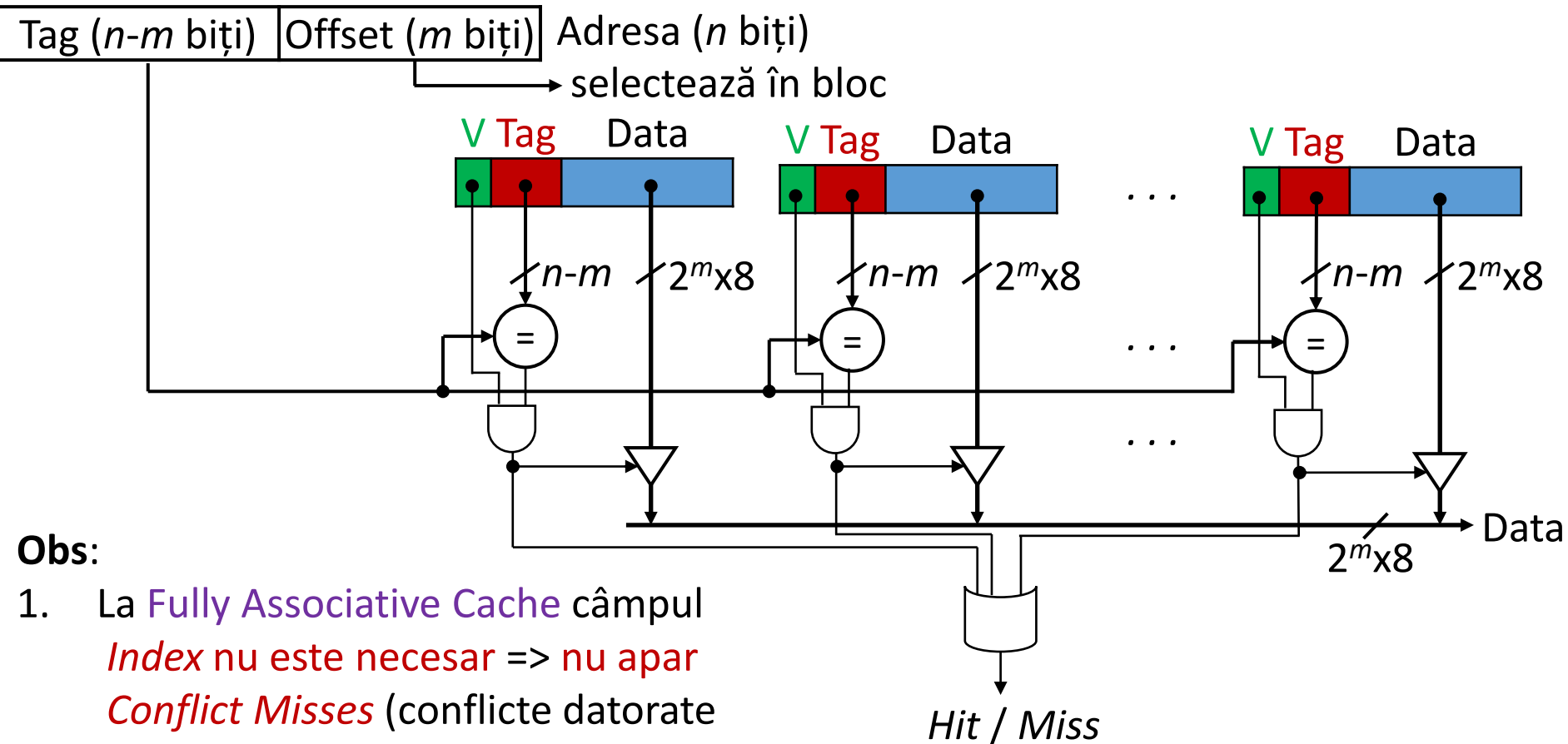
63	19 18	4 3	2 1 0
Tag	Index	Block offset	Byte offset

Ex₁: 4-way Set Associative de 4KB, cu 256 (2⁸) seturi x 4 căi/set x 1 cuvânt/bloc
 1 cuvânt = 4 Bytes



11.2.2 Identificarea blocului (cont)

11.2.2.4 Identificarea blocului la Fully Associative Cache

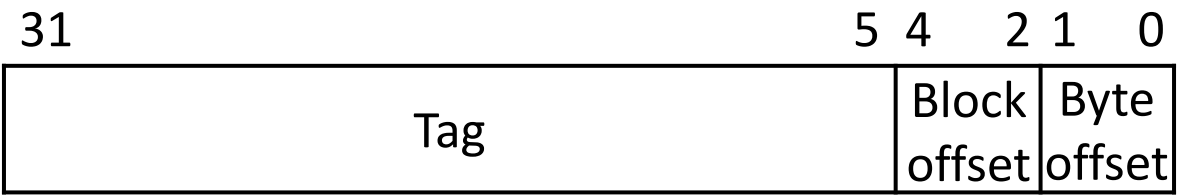


- Obs:**
- 1. La **Fully Associative Cache** câmpul **Index** nu este necesar => nu apar **Conflict Misses** (conflicte datorate aceluiași **Index** în adresă).
 - 2. Păstrează avantajele și dezavantajele enumerate la **Set Associative Cache**.
 - 3. Necesită multe comparatoare, motiv pentru care se implementează cu memorie asociativă (**Content Addressable Memory – CAM**), care folosește un tabel de căutare rapidă, ce returnează blocul din cache pe baza câmpului Tag.

Problemă: Care este dimensiunea câmpurilor dintr-o adresă de 32 biți dacă se folosește Fully Associative Cache de 2KB cu 8 cuvinte / bloc?

Rezolvare:

Dim. bloc = 8 x 4 Bytes = 2⁵ Bytes => Dim. Offset = 5 biți din care 2 sunt Byte offset.
Dim. Tag = Dim. adresă – Dim. Offset = 32 – 5 = 27 biți

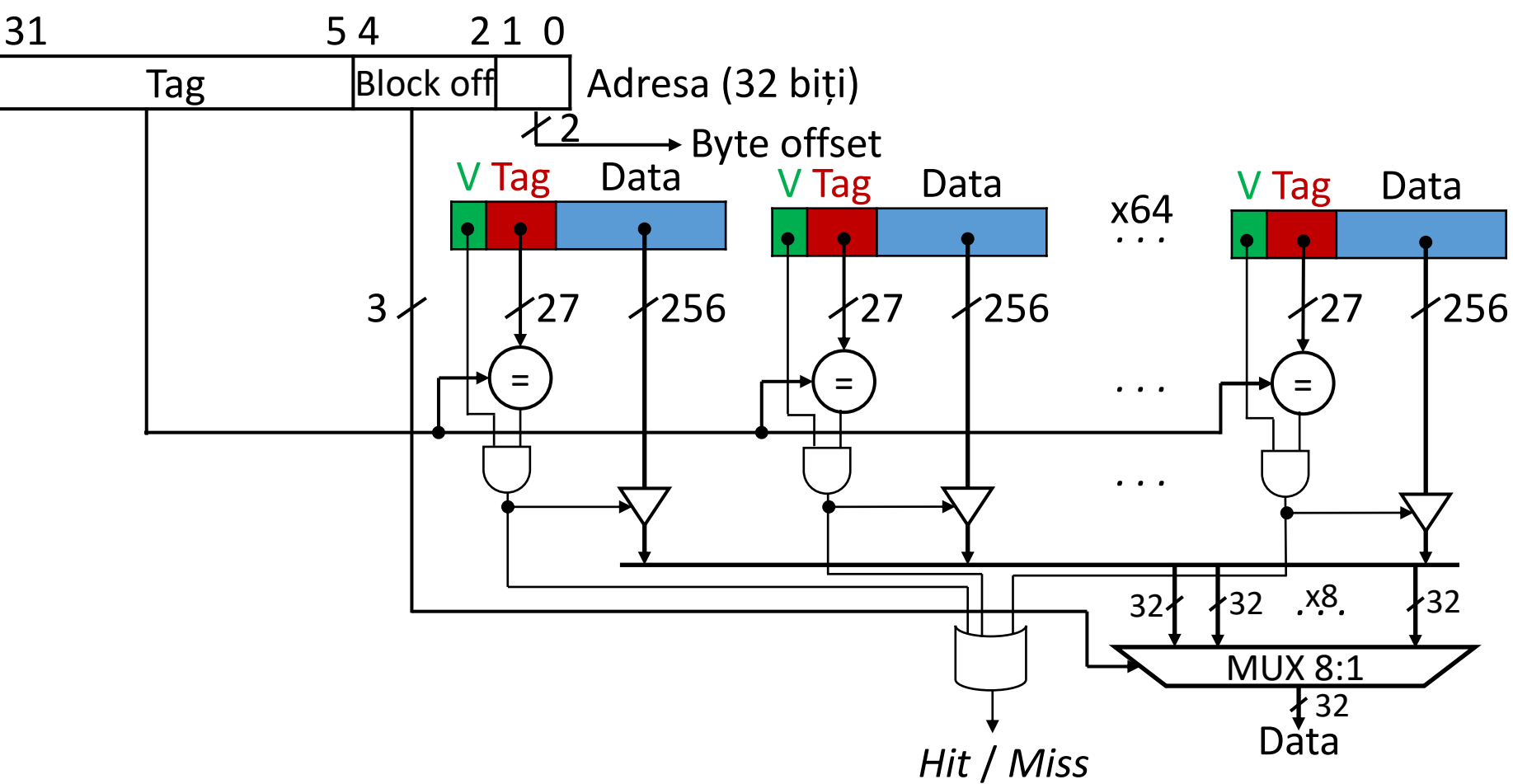


Câte căi sunt?

Nr. căi = Dim. cache / Dim. bloc = 2¹¹ / 2⁵ = 2⁶ = 64

Dimensiune reală = 64 x (1+27+32x8) biți = 64 x 284/8 Bytes = 2272 Bytes = 2.21875KB

Problemă: Proiectarea memoriei



11.2.3 Înlocuirea blocului la *Miss*

- La **Direct Mapped cache** – având 1 bloc/locație => nu se poate alege, întotdeauna este înlocuit blocul din locația accesată.
- La **Set/Fully Associative cache** – având mai multe blocuri pe set înlocuirea se face numai când nu mai sunt blocuri libere în setul accesat, folosind un algoritm implementat în hardware:
 - ❑ **Least Recently Used (LRU)** – se înlocuiește blocul din set utilizat cel mai demult.
 - Implementare facilă la 2-way Set Associative: 1 bit asociat blocului, denumit *USE*, se setează la 1 când blocul este accesat, iar bitul celui alt bloc din set este resetat la 0 => se înlocuiește blocul cu *USE*=0.
 - Necesită implementare mai complexă la asociativitate superioară.
 - ❑ **First-In-First-Out (FIFO)** – se înlocuiește blocul cel mai vechi din set.
 - Se folosește un algoritm de tip *round robin* (buffer circular cu **Head-Tail**).
 - ❑ **Least Frequently Used (LFU)** – se înlocuiește blocul din set cu cele mai puține accesări.
 - Un numărător, asociat fiecărui bloc, este incrementat la fiecare accesare => numărătorul cu valoarea minimă indică blocul de înlocuit.
 - ❑ **Random** – blocul din set înlocuit se alege pseudo-aleator.
 - Ușor de implementat în hardware (registru de deplasare + 1 XOR), această metodă oferă performanțe doar puțin mai slabe ca cele de mai sus.

Problema₁: Să se determine rezoluția *Hit/Miss* la următorul set de adrese de citire din memorie știind că se folosește un **Direct Mapped Cache** cu 4 blocuri de 1 semi-cuvânt pe 2 Bytes: 12, 20, 12, 8, 20. Adresarea se face la nivel de Byte.

Rezolvare:

Dim. bloc = 2 Bytes => $Bl\# = \lfloor Adresa / 2 \rfloor$ (împărțire întreagă)

Index bloc = $\text{Bl\# \% Cache_Blocks_No} = \text{Bl\# \% 4}$

$$\text{Tag bloc} = \lfloor \text{Bl\#} / \text{Cache_Blocks_No} \rfloor = \lfloor \text{Bl\#} / 4 \rfloor$$

Adr	Bl#	Tag	Idx
12	6	1	2
20	10	2	2
12	6	1	2
8	4	0	0
20	10	2	2

	Adr	Bl#	Tag	Idx	H/M
1	12	6	1	2	M

Direct Mapped

	V	Tag	Data
0	0		
1	0		
2	1	1	Bl#6
3	0		

Încarcă Bl#6 în cache.

	Adr	Bl#	Tag	Idx	H/M
	12	6	1	2	M
	20	10	2	2	M
3	12	6	1	2	M

	V	Tag	Data
0	0		
1	0		
2	1	1	BI#6
3	0		

Înlocuieste Bl#10 cu BL#6.

	Adr	Bl#	Tag	Idx	H/M
	12	6	1	2	M
2	20	10	2	2	M

Direct Mapped

	V	Tag	Data
0	0		
1	0		
2	1	2	B1#10
3	0		

Înlocuiește Bl#6 cu BL#10.

	Adr	Bl#	Tag	Idx	H/M
4	12	6	1	2	M
	20	10	2	2	M
	12	6	1	2	M
	8	4	0	0	M
5	20	10	2	2	M

	V	Tag	Data
0	1	0	BI#4
1	0		
2	1	2	BI#10
3	0		

Încarcă BL#4 si înlocuieste Bl#6 cu BL#10.

Problema₂: Să se determine rezoluția *Hit/Miss* la următorul set de adrese de citire din memorie știind că se folosește un **2-way Set Associative Cache** cu 4 blocuri de 1 semi-cuvânt pe 2 Bytes: 12, 20, 12, 8, 20. Adresarea se face la nivel de Byte și se folosește LRU la înlocuire.

Rezolvare:

Dim. bloc = 2 Bytes => **Bl#** = $\lfloor \text{Adresa} / 2 \rfloor$ (împărțire întreagă)

Index set = $\text{Bl\#} \% \text{Cache_Sets_No} = \text{Bl\#} \% 2$

Tag bloc = $\lfloor \text{Bl\#} / \text{Cache_Sets_No} \rfloor = \lfloor \text{Bl\#} / 2 \rfloor$

Adr	Bl#	Tag	Idx
12	6	3	0
20	10	5	0
12	6	3	0
8	4	2	0
20	10	5	0

1

2-way Set Associative

Adr	Bl#	Tag	Idx	H/M
12	6	3	0	M

V	Tag	Data	V	Tag	Data
0	1	3	Bl#6	0	
1	0		0		

Încarcă Bl#6 în cache.

Adr	Bl#	Tag	Idx	H/M
12	6	3	0	M
20	10	5	0	M
12	6	3	0	H

V	Tag	Data	V	Tag	Data
0	1	3	Bl#6	1	5
1	0		0		

LRU

3

2

2-way Set Associative

Adr	Bl#	Tag	Idx	H/M
12	6	3	0	M
20	10	5	0	M

V	Tag	Data	V	Tag	Data
0	1	3	Bl#6	1	5
1	0		0		

Încarcă Bl#10 în cache.

LRU

Adr	Bl#	Tag	Idx	H/M
12	6	3	0	M
20	10	5	0	M
12	6	1	0	H
8	4	2	0	M
20	10	5	0	M

V	Tag	Data	V	Tag	Data
0	1	3	Bl#6	1	2
1	0		0		

LRU

V	Tag	Data	V	Tag	Data
0	1	5	Bl#10	1	2
1	0		0		

4, 5

Înlocuiește Bl#10 cu Bl#4 și apoi Bl#6 cu Bl#10. **LRU**

Problema₃: Să se determine rezoluția *Hit/Miss* la următorul set de adrese de citire din memorie știind că se folosește un **Fully Associative Cache** cu 4 blocuri de 1 semi-cuvânt pe 2 Bytes: 12, 20, 12, 8, 20. Adresarea se face la nivel de Byte și se folosește LRU la înlocuire.

Rezolvare:

Dim. bloc = 2 Bytes => **Bl#** = $\lfloor \text{Adresa} / 2 \rfloor$ (împărțire întreagă)

Tag bloc = $\lfloor \text{Bl\#} / \text{Cache_Sets_No} \rfloor = \lfloor \text{Bl\#} / 1 \rfloor = \text{Bl\#}$

Adr	Bl#	Tag
12	6	6
20	10	10
12	6	6
8	4	4
20	10	10

Fully Associative

	Adr	Bl#	Tag	H/M
1	12	6	6	M

V	Tag	Data	V	Tag	Data	V	Tag	Data	V	Tag	Data
1	6	Bl#6	0			0			0		

Încarcă Bl#6 în cache.

	Adr	Bl#	Tag	H/M
2	20	10	10	M
3	12	6	6	H

V	Tag	Data	V	Tag	Data	V	Tag	Data	V	Tag	Data
1	6	Bl#6	1	10	Bl#10	0			0		

Încarcă Bl#10 în cache.

	Adr	Bl#	Tag	H/M
4	8	4	4	M
5	20	10	10	H

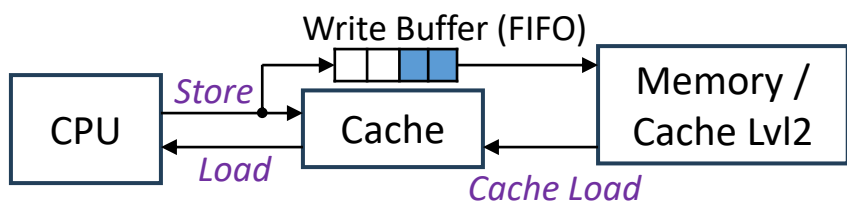
V	Tag	Data	V	Tag	Data	V	Tag	Data	V	Tag	Data
1	6	Bl#6	1	10	Bl#10	1	4	Bl#4	0		

Încarcă Bl#4 în cache.

11.2.4 Modalități de scriere

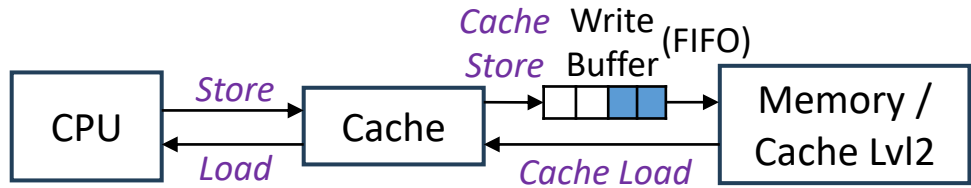
- Scrierea este mai lentă decât citirea deoarece se realizează după testul *Hit / Miss* (blocul scris este în cache?).
- Strategii de scriere uzuale:

1. **Write-Through**: Se scrie în blocul din cache și în memoria de nivel inferior.



- **Avantaj**: Se scrie necesarul instr., nu tot blocul. Asigură coerența datelor dintre cache și memorie => dacă se înlocuiește un bloc din cache la *Miss*, acesta nu mai trebuie scris în memoria de nivel inferior deoarece datele din el sunt corelate și acolo.
- **Dezavantaj**: Tehnica este lentă din cauza scrierii în memoria de nivel inferior, care este mai costisitoare ca la cache. **Soluție**: Sarcina scrierii în memorie este preluată de un *Write Buffer* (memorie FIFO) => procesorul trimite datele la *Write Buffer* (dacă are loc liber) și nu mai așteaptă scrierea lor decât dacă acesta se umple.

2. **Write-Back**: Se scrie doar în blocul din cache, iar scrierea datelor în memoria de nivel inferior se face numai la înlocuirea blocului (la *Miss*).



- Pt. a evita scrieri inutile la înlocuirea unui bloc, se asociază acestuia un **Dirty Bit**, care indică dacă s-a scris anterior în el; dacă blocul este “dirty” la înlocuire, trebuie scris în memorie pt. coerența datelor. **Notă**: Pt. viteză, scrierea se face cu *Write Buffer*.
- **Avantaj**: scrierea este mai rapidă fiindcă se face numai în cache.
- **Dezavantaj**: mai complex de implementat în hardware.

3. **Write-Once**: Prima scriere se face cu **Write-Through**, iar cele ulterioare cu **Write-Back**.

11.2.4 Modalități de scriere (cont)

Abordări în caz de *Miss* la scriere:

1. **Write allocate**: Blocul în care se face scriere este adus în cache din memoria de nivel inferior.
 - De obicei **se combină cu Write-Back** => după aducerea blocului în cache se scrie în el și se marchează “dirty”.
2. **No-write allocate**: Blocul în care se face scriere nu este adus în cache din memoria de nivel inferior.
 - De obicei **se combină cu Write-Through** => blocul nefiind adus în cache, scrierea se face doar în memoria de nivel inferior.

11.2.5 Performanța

Metrice de performanță uzuale:

- Timpul de execuție (CPU_{time}):

$$CPU_{time} = (\text{CPU clock cycles} + \text{Memory stall clock cycles}) \times \text{Clock cycle time}$$

include ciclurile pt. execuția programului
și pt. accesarea memoriei în caz de *Hit*

include penalizările în caz de *Miss* =
 $Miss\ rate \times Nr.\ accesuri\ la\ memorie \times Miss\ penalty$

- Average Memory Access Time (AMAT):

$$AMAT = Hit\ time + Miss\ rate \times Miss\ penalty$$

- Pentru îmbunătățirea AMAT se urmărește optimizarea celor 3 componente ale sale: *Hit time*, *Miss rate*, *Miss penalty*.

11.2.5 Performanța (cont)

Cauze pentru *Misses* – clasificarea *Misses* după modelul celor 3 “C”:

- **Compulsory** – *Miss* care apare implicit la prima accesare a unui bloc deoarece nu se află în cache. Apare chiar dacă dimensiunea cache este infinită. Numit *cold start Miss* (pornire la rece).
- **Capacity** – *Miss* care apare când programul folosește o zonă de memorie ce depășește capacitatea cache-ului. Apare chiar dacă am avea **Fully Associative Cache** cu strategie de înlocuire perfectă.
- **Conflict** – *Miss* care apare datorită conflictului pe aceeași locație la **Direct Mapped Cache** și la **Set Associative Cache**, dar care **nu ar apărea la Fully Associative Cache**.

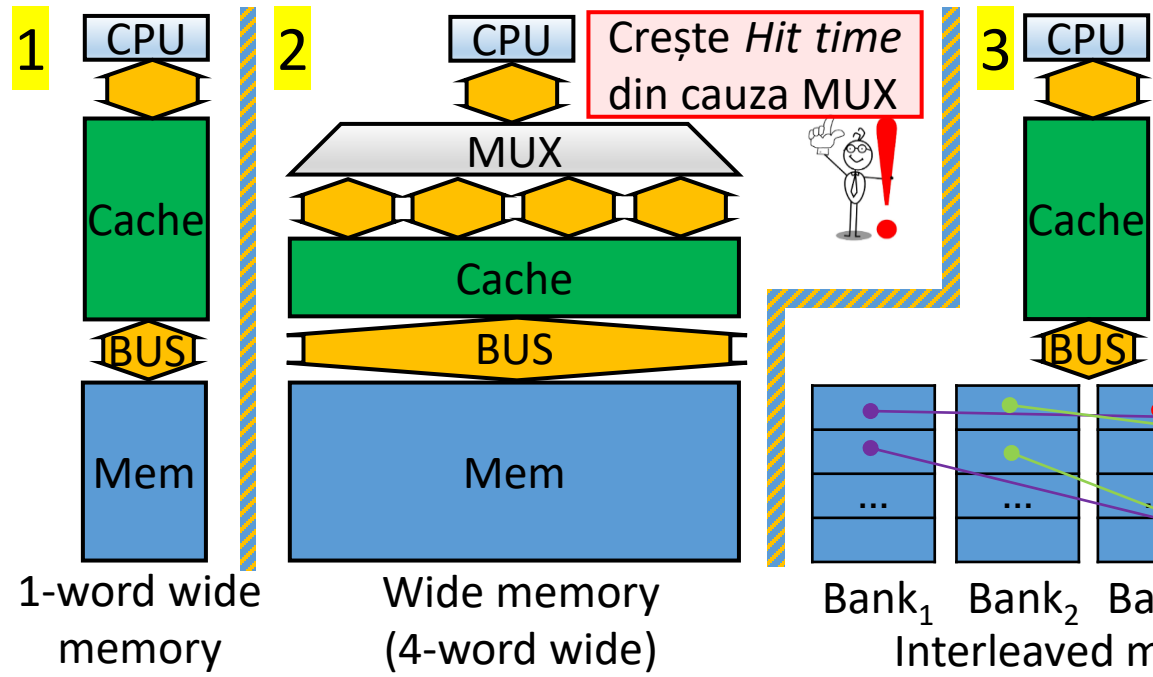
Al 4-lea “C”: **Coherence** – *Miss* care **apare la arhitecturi multiprocesor** datorită golirii regulate a memoriilor cache pentru a forța păstrarea coerenței între ele.

Strategii de îmbunătățire	Efectul asupra <i>Miss rate</i>	Posibile efecte negative
Creșterea capacității memoriei cache	<i>Miss rate</i> scade odată cu numărul de Capacity Misses .	Creșterea timpului de acces <i>Hit time</i> .
Creșterea asociativității	<i>Miss rate</i> scade odată cu numărul de Conflict Misses .	Creșterea timpului de acces <i>Hit time</i> . Necesită resurse hardware suplimentare.
Creșterea dimensiunii blocului	<i>Miss rate</i> scade datorită localității spațiale. Scade și Compulsory Misses .	Crește <i>Miss penalty</i> . La blocuri mari crește și <i>Miss rate</i> fiindcă scade asociativitatea.

11.2.6 Conexiunea cu memoria

Presupunem că avem un cache cu blocuri de 4 cuvinte (4x32 biți) și 3 cazuri de interconectare:

- 1. Memorie de 1 cuvânt / locație + BUS de lățime 1 cuvânt
- 2. Wide memory de 4 cuvinte / locație + BUS de lățime 4
- 3. Interleaved memory + BUS de lățime 1 cuvânt



Timpi de acces la memorie:

- 1 ciclu pt. trimiterea adresei
- 15 cicluri pt. accesul la date
- 1 ciclu răspuns cu datele citite

Miss penalty = citirea a 4 cuvinte (1 bloc) din memorie (în cicluri):

Caz₁: $1 + 4 \times (15 + 1) = 65$ cicluri

Caz₂: $1 + 1 \times 15 + 1 \times 1 = 17$ cicluri

Caz₃: $1 + 1 \times 15 + 4 \times 1 = 20$ cicluri



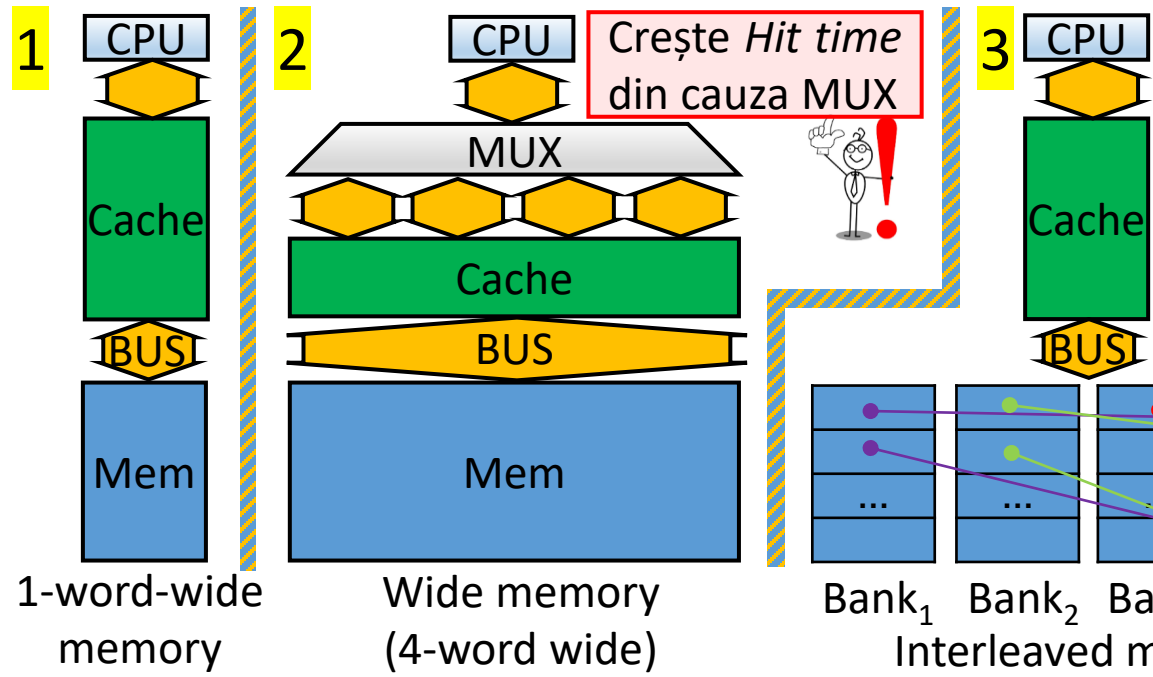
Interleaved memory (tehnologia actuală) = memorie în care Bank-urile conțin cuvinte (words) intercalate d.p.d.v. al locației. Bank-urile citesc și scriu concomitent => crește accesul de la 1 la 4 cuvinte / ciclu:

- Bank₁ conține cuvintele 0, 4, 8, ...
- Bank₂ conține cuvintele 1, 5, 9, ...
- Etc.

11.2.6 Conexiunea cu memoria

Presupunem că avem un cache cu blocuri de 4 cuvinte (4x32 biți) și 3 cazuri de interconectare:

- 1. Memorie de 1 cuvânt / locație + BUS de lățime 1 cuvânt
- 2. Wide memory de 4 cuvinte / locație + BUS de lățime 4
- 3. Interleaved memory + BUS de lățime 1 cuvânt



Timpi de acces la memorie:

- 1 ciclu pt. trimiterea adresei
- 15 cicluri pt. accesul la date
- 1 ciclu răspuns cu datele citite

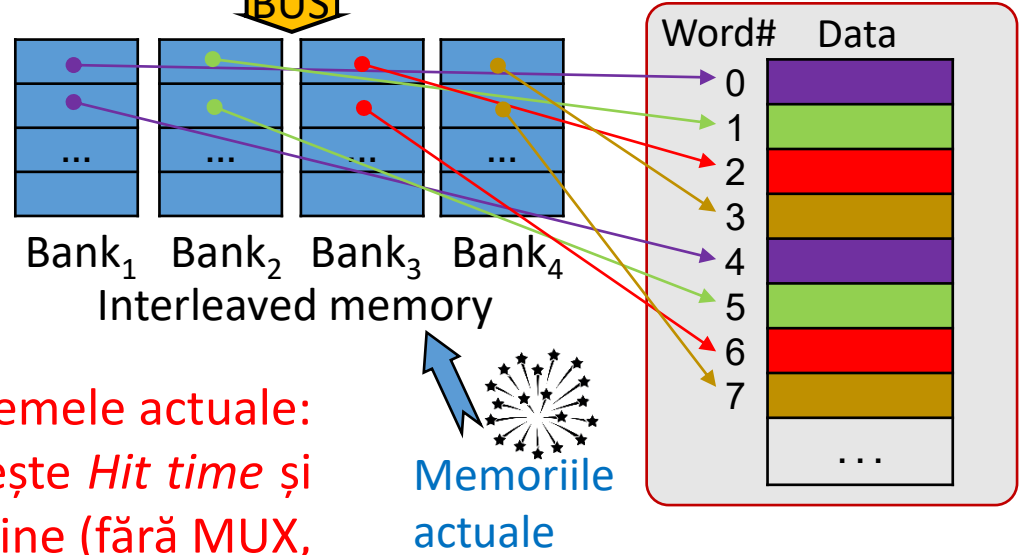
Miss penalty = citirea a 4 cuvinte (1 bloc) din memorie (în cicluri):

Caz₁: 1 + 4 x (15 + 1) = 65 cicluri

Caz₂: 1 + 1x15 + 1x1 = 17 cicluri

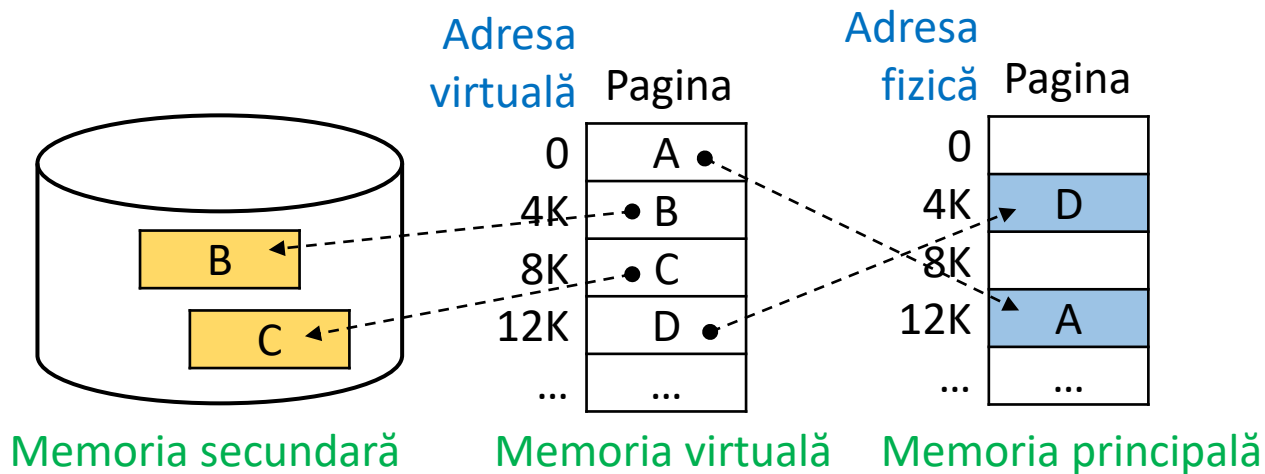
Caz₃: 1 + 1x15 + 4x1 = 20 cicluri

Varianta 3 este cea preferată în sistemele actuale: are Miss penalty relativ mic, nu crește Hit time și folosește resurse hardware mai puține (fără MUX, BUS-ul este mai îngust).



11.3 Memoria virtuală

- **Spațiul de adrese virtuale** = spațiul adresabil de programe. Este **determinat de ISA**.
- **Motivație**: Spațiul adreselor virtuale $\geq$ Capacitatea memoriei principale (mem. fizică).
- Organizatoric, memoria virtuală = colecție de **blocuri de mărime fixă** (ex. 4KB), numite **pagini**. **Notă**: Pagini consecutive în memoria virtuală au echivalent pagini distribuite la diverse locații în memoria principală și cea secundară (ex. discul magnetic, memoria flash).
Exemplu:



- **Avantaje**:
 - Iluzie de **memorie fizică (principală) mare** – fiecare program/proces are la dispoziție tot spațiul de adrese virtuale.
 - **Relocarea programelor** – la pornire, programelor li se alocă pagini fizice în memoria principală și secundară, de către sistemul de operare. Paginile în memoria secundară sunt alocate într-un spațiu rezervat, numit **swap space**.
 - **Protecție** – chiar dacă programele/procese executate concurent accesează aparent același spațiu de adrese virtuale, în realitate paginile accesate fizic sunt diferite.

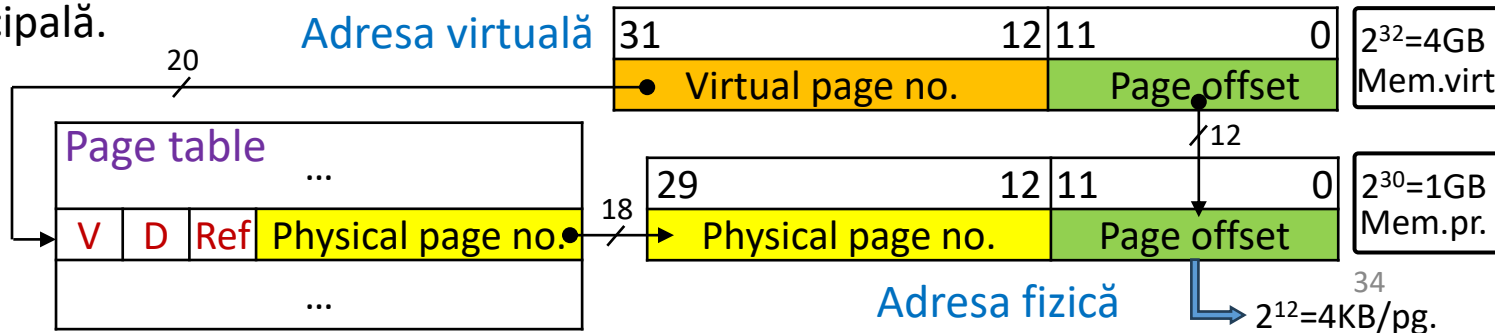
11.3 Memoria virtuală

11.3.1 Definiție: Memoria virtuală = tehnică ce conferă memoriei principale **rolul de cache** pentru memoria secundară.  **Notă:** Unitatea de transfer între disc și memorie este pagina.

- Prezența unei pagini în memoria principală este indicată de un bit **V (Valid)** asociat.
- Absența unei pagini în memoria principală se numește **page fault** => pagina se va aduce de pe disc într-o pagină liberă din memorie, altfel va înlocui o pagină existentă.
- Timpul de acces la memoria secundară este mare $\Leftrightarrow$ *Miss penalty* mare. **Soluții:**
 - Politica de înlocuire trebuie să fie eficientă (pt. *Miss rate* redus) și rapidă => **LRU aproximativ**: un bit **Ref (Reference)** asociat paginii este pus pe 1 atunci când este accesată. La înlocuire, se alege una din paginile cu Ref=0. Periodic, sistemul de operare resetează biții **Ref** la toate paginile.
 - Paginile au dimensiune mare față de blocurile cache: $\geq 4\text{KB}$.
Se folosește **Write-Back** la scriere și se asociază un bit **D (Dirty bit)** la fiecare pagină.
 - Acces lent => un **page fault** poate fi **tratat software** de sistemul de operare.

Traducerea “adresă virtuală $\rightarrow$ adresă fizică” se face cu ajutorul unui tabel **Page table** stocat în memoria principală.

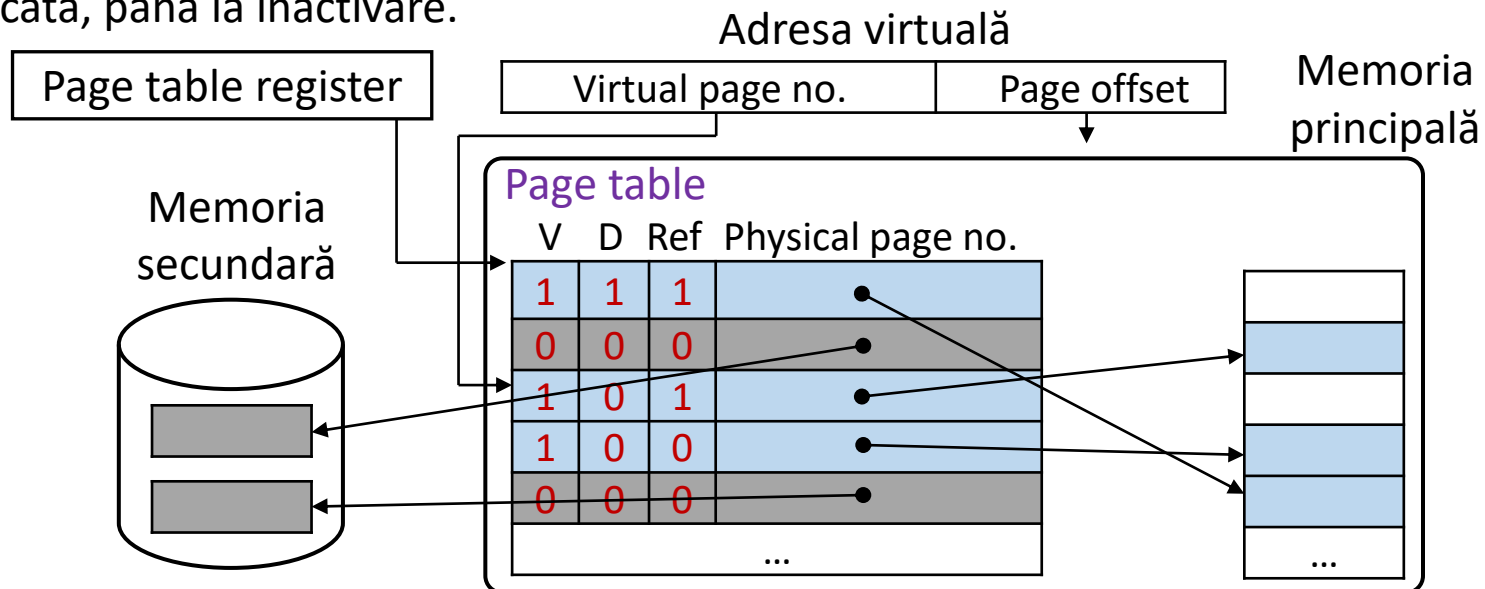
- Tabelul se indexează cu numărul paginii virtuale (Virtual page no.).



11.3 Memoria virtuală

11.3.2 Tabelul de pagini (Page table)

- Nu este un cache deoarece conține înregistrări pentru toate paginile virtuale.
- Fiecare înregistrare conține informații despre locația paginilor virtuale în memoria principală sau în memoria secundară, dar și biți de control precum: *Valid*, *Dirty*, *Reference*.
- Dacă pagina se află în memoria principală, se compune adresa fizică și se trimite la memorie (la cache-ul ei) pentru citirea datelor, altfel apare *Page fault* tratat de sistemul de operare.
- Fiecare proces are un *Page table* propriu, pentru administrarea memoriei sale virtuale, care este stocat în memoria principală la o adresă indicată de registrul *Page table register*. La execuție cu procese concurente, acestea au acces la procesor (sunt active) alternativ. Activarea unui proces implică actualizarea registrului *Page table register* cu adresa tabelului de pagini asociat lui. Astfel, procesul va opera pe memoria sa virtuală, în cuanta de timp alocată, până la inactivare.



11.3 Memoria virtuală

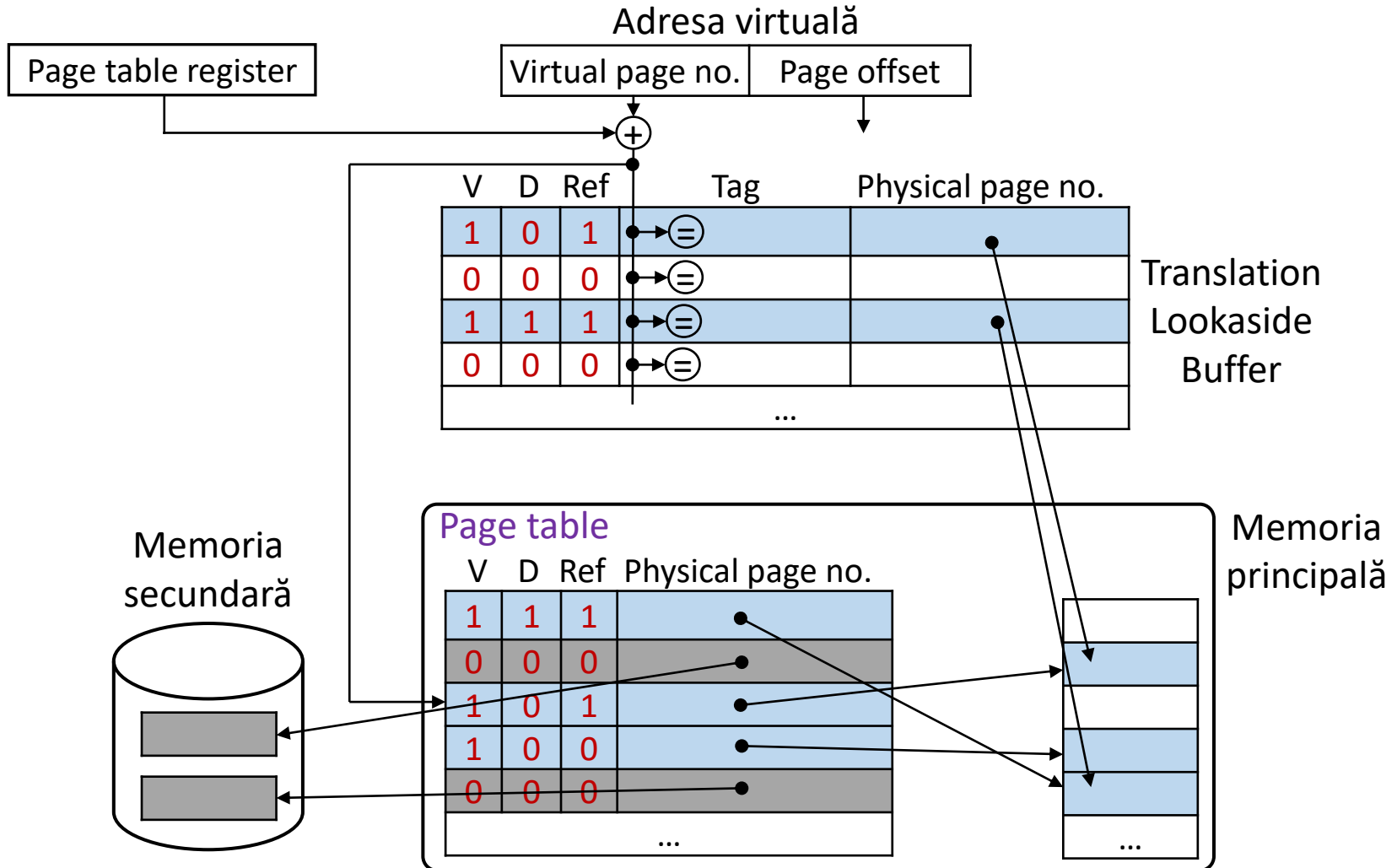
11.3.3 Cache pentru tabelul de pagini (**Translation-Lookaside Buffer – TLB**)

- **Motivație:** Utilizarea adresei virtuale presupune 2 accesuri la memoria principală:
 1. Accesul la *Page table* pt. traducere – este lent din cauza memoriei principale;
 2. Accesul la datele vizate de adresa fizică – este accelerat de cache-ul la memorie.
- **Soluție:** Se folosește un cache dedicat (sau integrat în cache-ul memoriei) și pentru *Page table*, numit **Translation-Lookaside Buffer (TLB)**, cu timp de acces redus: *Hit time* = 0.5-1 ciclu, *Miss penalty* = 10-100 cicluri.
- **Alte caracteristici pentru TLB:**
 - asociativitate ridicată, de regulă **Fully Associative** => *Miss rate* = 0.01-1%.
 - număr redus de intrări în cache: 16-512.
 - Mecanism de înlocuire **Random** fiind ușor de **implementat în hardware**.
 - Scriere cu **Write-Back**. **Notă:** La înlocuirea unei intrări *dirty* se salvează în memoria principală doar biții de control; adresa fizică nu se modifică, deci nu trebuie salvată.
 - Conține numai intrări pentru pagini aflate în memoria principală. Dacă o pagină este mutată în memoria secundară este eliminată și din TLB.

Notă: Un *TLB Miss* nu este echivalent cu *page fault*. De regulă *TLB Misses* apar mai frecvent ca *page faults* și se rezolvă mult mai rapid.

11.3 Memoria virtuală

11.3.4 Page table cu TLB Total Asociativ



- Cazul cel mai favorabil: *TLB Hit* (caz în care nu se mai verifică *page fault*) urmat de *Cache Hit*.
- Cazul cel mai nefavorabil: *TLB Miss* urmat de *page fault* și apoi de *Cache Miss*.

Probleme propuse

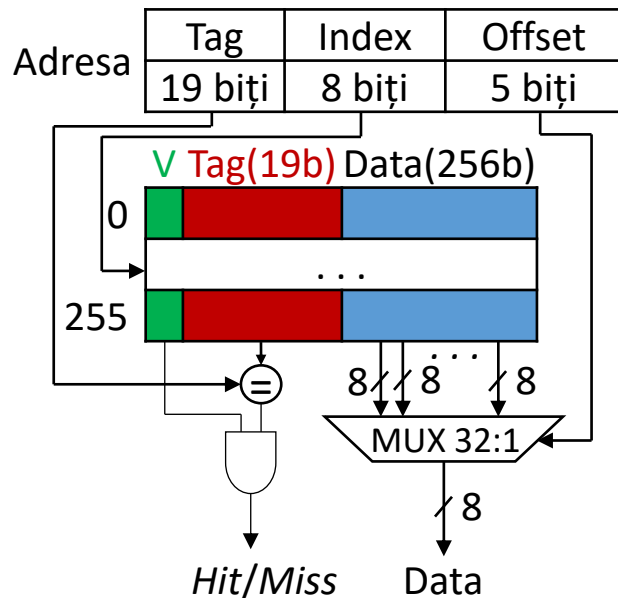
Un procesor generează adrese pe 32 de biți pentru o memorie adresabilă pe octet. Proiectați o memorie cache de 8KB (strict pentru biții de date, nu include biții de *Tag*). Dimensiunea unui bloc din cache este de 32 de octeți. Se va face schema memoriei cache și se va explica mecanismul de decodificare a adresei pentru:

- 1) Cache de tip Direct Mapped;
- 2) Cache de tip 4-way Set Associative.

Rezolvare:

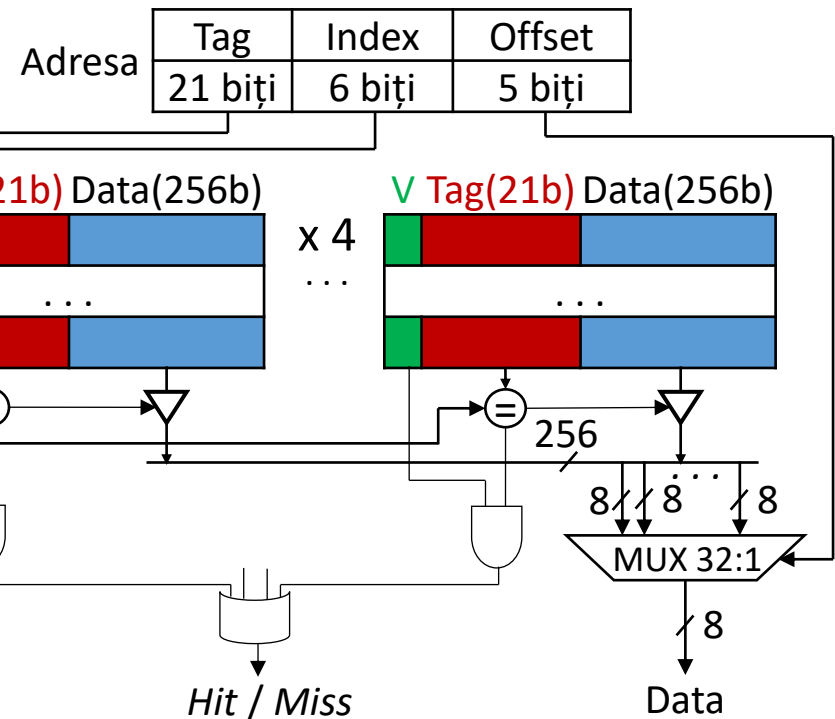
1) Direct Mapped

Dim. bloc = 32B = 2^5 B $\Rightarrow$ Offset = 5 biți
Nr. blocuri: $2^{13}/2^5 = 2^8 \Rightarrow$ Index = 8 biți
Dim. Tag = $32 - 8 - 5 = 19$ biți



2) 4-way Set Associative

Dim. bloc = 32B = 2^5 B $\Rightarrow$ Offset = 5 biți
Nr. seturi: $2^{13}/(2^5 \times 4) = 2^6 \Rightarrow$ Index = 6 biți
Dim. Tag = $32 - 6 - 5 = 21$ biți



Referințe

- [1] D. A. Patterson, J. L. Hennessy, “Computer Organization and Design: The Hardware/Software Interface”, 5th edition, Cap. 5.1-5.4, 5.7, 5.8, Ed. Morgan–Kaufmann, 2013.
- [2] J. L. Hennessy, D. A. Patterson, “Computer Architecture: A Quantitative Approach”, 5th edition, Cap. 2.1, Anexa B.1-B.3, Ed. Morgan–Kaufmann, 2011.